

Aficiones-NM

**Pepe Calderón, Fernando Sola, Daniel Ayala, Ana Belén
Sánchez, Inma Hernández, Margarita Cruz, Carlos Aré-
valo, David Ruiz**



Escuela Técnica Superior de
Ingeniería Informática

1. Requisitos de información	2
1.1. RI-1: Usuarios	2
1.2. RI-02: Aficiones	2
2. Requisitos funcionales	3
2.1. RF-03: Informes sobre aficiones	3
3. Diagrama de clases Aficiones-NM	4
4. Modelo relacional Aficiones-NM (tabla intermedia Usuario–Afición)	5
4.1. Álgebra relacional	6
5. Ejercicio Aficiones-NM	8
5.1. Script SQL para crear la base de datos	8
5.2. Script SQL para la carga inicial de datos	9
5.3. Consultas	11
5.4. SQL avanzado	12
5.5. Pruebas de aceptación	13
5.6. Transacciones	14
5.7. Pruebas SQL	14

Requisitos Este ejercicio amplía Usuarios con una relación N:M y un catálogo abierto de aficiones.

Requisitos de información

1.1. RI-1: Usuarios

- Mismo que ejercicio de Usuarios

1.2. RI-02: Aficiones

- Como: Profesor de la asignatura
- Quiero: Almacenar las aficiones de los usuarios en un catálogo abierto. Cada usuario puede tener varias aficiones o ninguna, y se pueden añadir nuevas aficiones sin modificar el esquema.
- Para: Que el estudiante tenga en cuenta este requisito en el modelo conceptual, relacional y tecnológico.

Requisitos funcionales

2.1. RF-03: Informes sobre aficiones

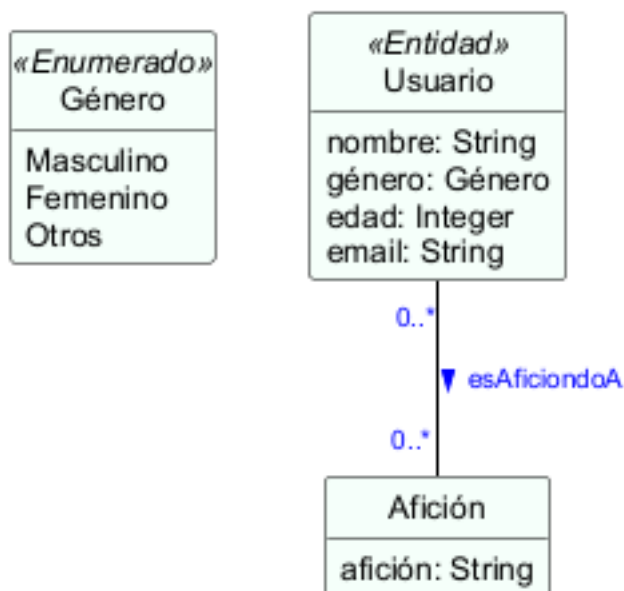
- Como: Profesor de la asignatura
- Quiero: Que el sistema sea capaz de generar los siguientes informes:
 - Listado de usuarios con sus aficiones.
 - Usuarios a los que le gusta el cine.
 - Usuarios que no tienen aficiones.
 - Número de aficiones de cada usuario.
 - Máximo número de aficiones que tiene un usuario.
 - Usuarios con el máximo número de aficiones.
- Para: Que el alumno realice consultas en Álgebra Relacional y SQL.

Modelo Conceptual

Diagrama de clases Aficiones-NM

Las aficiones se almacenan como entidad propia para permitir un catálogo abierto; la relación Usuario–Afición es N:M.

Aficiones-NM (variante dinámica)



Modelo Relacional

Modelo relacional Aficiones-NM (tabla intermedia Usuario–Afición)

```

Usuarios = { usuarioId, nombre, edad, género, email }
  PK(usuarioId)
  AK(email)
Aficiones = { aficionId, afición }
  PK(aficionId)
UsuariosAficiones = { usuarioAficionId, usuarioId, aficionId }
  PK(usuarioAficionId)
  FK(usuarioId) / Usuarios
  FK(aficiónId) / Aficiones
  AK(usuarioId, aficiónId)

Usuarios = {
  (u1, "David Ruiz", 45, MASCULINO, "druiz@us.es"),
  (u2, "Carlos Arévalo", 58, MASCULINO, "carevalo@us.es"),
  (u3, "Margarita Cruz", 58, FEMENINO, "mcruz@us.es"),
  (u4, "Inma Hernández", 35, FEMENINO, "inmahernandez@us.es"),
  (u5, "Alfonso Márquez", 35, MASCULINO, "amarquez@us.es"),
  (u6, "Daniel Ayala", 28, MASCULINO, "dayalal@us.es"),
  (u7, "Raquel Sampedro", 55, FEMENINO, "rsampedro@gmail.com"),
  (u8, "Marta López", 18, FEMENINO, "mlopez@mail.com"),
  (u9, "David Ruiz", 25, MASCULINO, "druiz@mail.com"),
  (u10, "Andrea Gómez", 27, NULL, "agomez@mail.es"),
  (u11, "Ernesto Murillo", 55, OTRO, "emurillo@correo.es")
}

Aficiones = {
  (a1, "Cine"), (a2, "Deporte"), (a3, "Jugar al fútbol"),
  (a4, "Hacer senderismo"), (a5, "Montar a caballo")
}

UsuariosAficiones = {
  (ua1, u1, a3), (ua2, u1, a4), (ua3, u2, a1),

```

```

(ua4, u2, a2), (ua5, u2, a3), (ua6, u4, a4),
(ua7, u4, a2), (ua8, u4, a1), (ua9, u5, a3),
(ua10, u6, a2), (ua11, u8, a3), (ua12, u8, a4),
(ua13, u8, a1), (ua14, u8, a2), (ua15, u9, a3),
(ua16, u9, a1), (ua17, u9, a2), (ua18, u10, a4),
(ua19, u11, a5)
}

```

4.1. Álgebra relacional

- Renombrado:

$$\rho_{U(uid, nu, ed, g, em)} (Usuarios)$$

$$\rho_{A(aid, af)} (Aficiones)$$

$$\rho_{UA(uaid, uid, aid)} (UsuariosAficiones)$$

- Usuarios con sus aficiones:

$$UAA \leftarrow U \bowtie UA \bowtie A$$

```
UAA = { uid, nu, ed, g, em, uaid, aid, af }
```

```

UAA = {
  (u1, "David Ruiz", 45, MASCULINO, "druiz@us.es", ua1, a3, "Jugar al fútbol"),
  (u1, "David Ruiz", 45, MASCULINO, "druiz@us.es", ua2, a4, "Hacer senderismo"),
  (u2, "Carlos Arévalo", 58, MASCULINO, "carevalo@us.es", ua3, a1, "Cine"),
  (u2, "Carlos Arévalo", 58, MASCULINO, "carevalo@us.es", ua4, a2, "Deporte"),
  (u2, "Carlos Arévalo", 58, MASCULINO, "carevalo@us.es", ua5, a3, "Jugar al fútbol"),
  (u4, "Inma Hernández", 35, FEMENINO, "inmahernandez@us.es", ua6, a4, "Hacer senderismo"),
  (u4, "Inma Hernández", 35, FEMENINO, "inmahernandez@us.es", ua7, a2, "Deporte"),
  (u4, "Inma Hernández", 35, FEMENINO, "inmahernandez@us.es", ua8, a1, "Cine"),
  (u5, "Alfonso Márquez", 35, MASCULINO, "amarquez@us.es", ua9, a3, "Jugar al fútbol"),
  (u6, "Daniel Ayala", 28, MASCULINO, "dayalal@us.es", ua10, a2, "Deporte"),
  (u8, "Marta López", 18, FEMENINO, "mlopez@mail.com", ua11, a3, "Jugar al fútbol"),
  (u8, "Marta López", 18, FEMENINO, "mlopez@mail.com", ua12, a4, "Hacer senderismo"),
  (u8, "Marta López", 18, FEMENINO, "mlopez@mail.com", ua13, a1, "Cine"),
  (u8, "Marta López", 18, FEMENINO, "mlopez@mail.com", ua14, a2, "Deporte"),
  (u9, "David Ruiz", 25, MASCULINO, "druiz@mail.com", ua15, a3, "Jugar al fútbol"),
  (u9, "David Ruiz", 25, MASCULINO, "druiz@mail.com", ua16, a1, "Cine"),
  (u9, "David Ruiz", 25, MASCULINO, "druiz@mail.com", ua17, a2, "Deporte"),
  (u10, "Andrea Gómez", 27, NULL, "agomez@mail.es", ua18, a4, "Hacer senderismo"),
  (u11, "Ernesto Murillo", 55, OTRO, "emurillo@correo.es", ua19, a5, "Montar a caballo")
}

```

- Usuarios a los que les gusta el cine:

$$UCine \leftarrow \Pi_{nu} \left(\sigma_{af = \text{Cine}} (UAA) \right)$$

4. Modelo relacional Aficiones-NM (tabla intermedia Usuario-Afición)

```
UCine = { nu }
```

```
UCine = {  
  ("Carlos Arévalo"),  
  ("Inma Hernández"),  
  ("Marta López"),  
  ("David Ruiz")  
}
```

- Usuarios sin aficiones:

$$USinAfi \leftarrow \Pi_{nu}(U \bowtie (\Pi_{uid}(U) - \Pi_{uid}(UA)))$$

```
USinAfi = { nu }
```

```
USinAfi = {  
  ("Margarita Cruz"),  
  ("Raquel Sampedro")  
}
```

- Usuarios con todas las aficiones:

$$UsuTodasAfi \leftarrow \Pi_{nu,uid} \left(\left(\Pi_{uid,aid}(UA) \div \Pi_{aid}(A) \right) \bowtie U \right)$$

```
UsuTodasAfi = { nu, uid }
```

```
UsuTodasAfi = {}
```

Modelo Tecnológico

Ejercicio Aficiones-NM

5.1. Script SQL para crear la base de datos

aficiones-nm/createDB.sql

```
--
-- Autor: David Ruiz
-- Fecha: Noviembre 2024
-- Descripción: Script para crear la BD de Aficiones Dinámicas
--

DROP DATABASE IF EXISTS HobbiesNMDB;
CREATE DATABASE HobbiesNMDB;
USE HobbiesNMDB;

-- =====
-- Eliminamos tablas previas
-- =====

SET FOREIGN_KEY_CHECKS = 0;
DROP TABLE IF EXISTS user_hobby_links;
DROP TABLE IF EXISTS hobbies;
DROP TABLE IF EXISTS users;
SET FOREIGN_KEY_CHECKS = 1;

-- =====
-- Tabla: users
```

```

-- =====
CREATE TABLE users (
    user_id INT AUTO_INCREMENT,
    full_name VARCHAR(64) NOT NULL,
    gender ENUM('MASCULINO', 'FEMENINO', 'OTRO') NULL,
    age TINYINT NOT NULL,
    email VARCHAR(255) NOT NULL,
    PRIMARY KEY (user_id),
    CONSTRAINT rn01_users_adult_age CHECK (age >= 18),
    CONSTRAINT rn02_users_unique_email UNIQUE (email)
);

-- =====
-- Tabla: hobbies
-- =====
CREATE TABLE hobbies (
    hobby_id INT AUTO_INCREMENT,
    hobby_name VARCHAR(128) NOT NULL,
    PRIMARY KEY (hobby_id)
);

-- =====
-- Tabla: user_hobby_links
-- =====
CREATE TABLE user_hobby_links (
    user_hobby_id INT AUTO_INCREMENT,
    user_id INT NOT NULL,
    hobby_id INT NOT NULL,
    PRIMARY KEY (user_hobby_id),
    CONSTRAINT fk_user_hobby_links_user FOREIGN KEY (user_id) REFERENCES
users(user_id),
    CONSTRAINT fk_user_hobby_links_hobby FOREIGN KEY (hobby_id) REFERENCES
hobbies(hobby_id),
    CONSTRAINT rn03_user_hobby_unique UNIQUE (user_id, hobby_id)
);

```

5.2. Script SQL para la carga inicial de datos

aficiones-nm/populateDB.sql

```

--
-- Autor: David Ruiz
-- Fecha: Noviembre 2024
-- Descripción: Procedimiento para poblar la BD de Aficiones Dinámicas
--
USE HobbiesNMDB;

```

```

DELIMITER //
CREATE OR REPLACE PROCEDURE p_populate()
BEGIN
    DELETE FROM user_hobby_links;
    DELETE FROM hobbies;
    DELETE FROM users;

    ALTER TABLE users AUTO_INCREMENT = 1;
    INSERT INTO users (full_name, gender, age, email) VALUES
        ('David Ruiz', 'MASCULINO', 45, 'druiz@us.es'),
        ('Carlos Arévalo', 'MASCULINO', 58, 'carevalo@us.es'),
        ('Margarita Cruz', 'FEMENINO', 58, 'mcruz@us.es'),
        ('Inma Hernández', 'FEMENINO', 35, 'inmahernandez@us.es'),
        ('Alfonso Márquez', 'MASCULINO', 35, 'amarquez@us.es'),
        ('Daniel Ayala', 'MASCULINO', 28, 'dayalal@us.es'),
        ('Raquel Sampedro', 'FEMENINO', 55, 'rsampedro@gmail.com'),
        ('Marta López', 'FEMENINO', 18, 'mlopez@mail.com'),
        ('David Ruiz', 'MASCULINO', 25, 'druiz@mail.com');

    INSERT INTO users (full_name, age, email)
        VALUES ('Andrea Gómez', 42, 'agomez@mail.es');

    INSERT INTO users (full_name, gender, age, email)
        VALUES ('Ernesto Murillo', 'OTRO', 55, 'emurillo@correo.es');

    ALTER TABLE hobbies AUTO_INCREMENT = 1;
    INSERT INTO hobbies (hobby_name) VALUES
        ('Leer'),
        ('Ir al cine'),
        ('Hacer deporte'),
        ('Hacer de comer'),
        ('Montar a caballo');

    ALTER TABLE user_hobby_links AUTO_INCREMENT = 1;
    INSERT INTO user_hobby_links (user_id, hobby_id) VALUES
        (1,3), (1,4),
        (2,3), (2,1), (2,2),
        (4,4), (4,2), (4,1),
        (5,3),
        (6,2),
        (8,3), (8,4), (8,1), (8,2),
        (9,3), (9,1), (9,2),
        (10,4),
        (11,5);
END //
DELIMITER ;

CALL p_populate();

```

5.3. Consultas

aficiones-nm/queries.sql

```
--
-- Autor: David Ruiz
-- Fecha: Noviembre 2024
-- Descripción: Consultas de referencia para Aficiones Dinámicas
--

USE HobbiesNMDB;

-- Usuarios con sus aficiones
SELECT u.user_id, l.user_hobby_id, u.full_name, u.age, u.email,
h.hobby_name
FROM users u
JOIN user_hobby_links l ON u.user_id = l.user_id
JOIN hobbies h ON h.hobby_id = l.hobby_id;

CREATE OR REPLACE VIEW v_user_hobbies AS
SELECT u.user_id, l.user_hobby_id, u.full_name, u.age, u.email,
h.hobby_name
FROM users u
JOIN user_hobby_links l ON u.user_id = l.user_id
JOIN hobbies h ON h.hobby_id = l.hobby_id;

-- Usuarios a los que les gusta el cine
SELECT full_name, age, email
FROM v_user_hobbies
WHERE hobby_name LIKE '%cine%';

-- Usuarios sin aficiones
SELECT *
FROM users u
WHERE NOT EXISTS (
    SELECT 1 FROM user_hobby_links l WHERE l.user_id = u.user_id
);

-- Número de aficiones por usuario
CREATE OR REPLACE VIEW v_user_hobby_count AS
SELECT user_id, full_name, email, COUNT(*) AS total
FROM v_user_hobbies
GROUP BY user_id;

SELECT * FROM v_user_hobby_count;

-- Máximo número de aficiones
SELECT MAX(total) AS max_hobbies FROM v_user_hobby_count;

-- Usuarios con el máximo número de aficiones
```

```
SELECT * FROM v_user_hobby_count
WHERE total = (SELECT MAX(total) FROM v_user_hobby_count);
```

5.4. SQL avanzado

aficiones-nm/procedures.sql

```
--
-- Autor: David Ruiz
-- Fecha: Noviembre 2024
-- Descripción: Procedimientos y funciones para Aficiones Dinámicas
--

USE HobbiesNMDB;

-- Insertar usuario
DELIMITER //
CREATE OR REPLACE PROCEDURE p_insert_user(
    IN p_full_name VARCHAR(64),
    IN p_gender ENUM('MASCULINO', 'FEMENINO', 'OTRO'),
    IN p_age INT,
    IN p_email VARCHAR(255)
)
BEGIN
    INSERT INTO users (full_name, gender, age, email)
    VALUES (p_full_name, p_gender, p_age, p_email);
END //
DELIMITER ;

-- Insertar usuario y afición en una transacción
DELIMITER //
CREATE OR REPLACE PROCEDURE p_insert_user_with_hobby(
    IN p_full_name VARCHAR(64),
    IN p_gender ENUM('MASCULINO', 'FEMENINO', 'OTRO'),
    IN p_age INT,
    IN p_email VARCHAR(255),
    IN p_hobby_name VARCHAR(128)
)
BEGIN
    DECLARE v_user_id INT;
    DECLARE v_hobby_id INT;

    CALL p_insert_user(p_full_name, p_gender, p_age, p_email);
    SET v_user_id = LAST_INSERT_ID();

    INSERT INTO hobbies (hobby_name) VALUES (p_hobby_name);
```

```

    SET v_hobby_id = LAST_INSERT_ID();

    INSERT INTO user_hobby_links (user_id, hobby_id) VALUES (v_user_id,
v_hobby_id);
END //
DELIMITER ;

-- Versión transaccional
DELIMITER //
CREATE OR REPLACE PROCEDURE p_insert_user_with_hobby_tx(
    IN p_full_name VARCHAR(64),
    IN p_gender ENUM('MASCULINO', 'FEMENINO', 'OTRO'),
    IN p_age INT,
    IN p_email VARCHAR(255),
    IN p_hobby_name VARCHAR(128)
)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error en inserción de
usuario/afición';
    END;

    START TRANSACTION;
    CALL p_insert_user_with_hobby(p_full_name, p_gender, p_age,
p_email, p_hobby_name);
    COMMIT;
END //
DELIMITER ;

```

Realice un procedimiento para insertar en la tabla de usuarios e implemente la siguiente prueba de aceptación:

5.5. Pruebas de aceptación

PA-001: Usuarios

1. ☑ Insertar un usuario con datos correctos.
2. ☑ Insertar un usuario sin género.
3. ✗ Insertar un usuario con un email repetido.
4. ✗ Insertar un usuario menor de edad.

5.6. Transacciones

Realice un procedimiento para insertar una afición a un usuario nuevo, es decir, que inserte en las tres tablas:

Realice el mismo procedimiento pero de forma transaccional:

5.7. Pruebas SQL

aficiones-nm/tests.sql

```
--
-- Autor: David Ruiz
-- Fecha: Noviembre 2024
-- Descripción: Tests negativos para la BD de Aficiones Dinámicas
--
USE HobbiesNMDB;

-- =====
-- TABLA DE RESULTADOS
-- =====

CREATE OR REPLACE TABLE test_results (
    test_id VARCHAR(20) PRIMARY KEY,
    test_name VARCHAR(200) NOT NULL,
    test_message VARCHAR(500) NOT NULL,
    test_status ENUM('PASS', 'FAIL', 'ERROR') NOT NULL,
    execution_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- =====
-- PROCEDIMIENTO AUXILIAR
-- =====

DELIMITER //
CREATE OR REPLACE PROCEDURE p_log_test(
    IN p_test_id VARCHAR(20),
    IN p_message VARCHAR(500),
    IN p_status ENUM('PASS', 'FAIL', 'ERROR')
)
BEGIN
    INSERT INTO test_results(test_id, test_name, test_message, test_status)
    VALUES (p_test_id, SUBSTRING_INDEX(p_message, ':', 1), p_message,
p_status);
END //
DELIMITER ;

-- =====
-- TESTS (RN01 - RN03 + FK)
-- =====
```

```

DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_rn01_age_adult()
BEGIN
    DECLARE EXIT HANDLER FOR SQLSTATE '23000'
        CALL p_log_test('RN01', 'RN01: Edad mínima 18 años', 'PASS');
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        CALL p_log_test('RN01', 'ERROR: No se produjo el error esperado
para la edad mínima', 'ERROR');

    CALL p_populate();

    INSERT INTO users (full_name, gender, age, email)
        VALUES ('Usuario Menor', 'MASCULINO', 17, 'menor@ejemplo.com');

    CALL p_log_test('RN01', 'ERROR: Se permitió un usuario menor de edad',
'FAIL');
END //
DELIMITER ;

DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_rn02_unique_email()
BEGIN
    DECLARE EXIT HANDLER FOR SQLSTATE '23000'
        CALL p_log_test('RN02', 'RN02: Email debe ser único', 'PASS');
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        CALL p_log_test('RN02', 'ERROR: No se produjo el error esperado
para el email duplicado', 'ERROR');

    CALL p_populate();

    INSERT INTO users (full_name, gender, age, email)
        VALUES ('Correo Duplicado', 'FEMENINO', 30, 'druiz@us.es');

    CALL p_log_test('RN02', 'ERROR: Se permitió duplicar un email',
'FAIL');
END //
DELIMITER ;

DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_rn03_unique_hobby_per_user()
BEGIN
    DECLARE EXIT HANDLER FOR SQLSTATE '23000'
        CALL p_log_test('RN03', 'RN03: No se pueden repetir aficiones por
usuario', 'PASS');
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        CALL p_log_test('RN03', 'ERROR: No se produjo el error esperado
para la afición repetida', 'ERROR');

    CALL p_populate();

```

```

INSERT INTO user_hobby_links (user_id, hobby_id) VALUES (1, 3);

CALL p_log_test('RN03', 'ERROR: Se permitió repetir afición para el
mismo usuario', 'FAIL');
END //
DELIMITER ;

DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_fk_user_exists()
BEGIN
    DECLARE EXIT HANDLER FOR SQLSTATE '23000'
        CALL p_log_test('FK01', 'FK01: user_id debe existir', 'PASS');
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        CALL p_log_test('FK01', 'ERROR: No se produjo el error esperado
para user_id', 'ERROR');

    CALL p_populate();

    INSERT INTO user_hobby_links (user_id, hobby_id) VALUES (999, 1);

    CALL p_log_test('FK01', 'ERROR: Se permitió FK user_id inexistente',
'FAIL');
END //
DELIMITER ;

DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_fk_hobby_exists()
BEGIN
    DECLARE EXIT HANDLER FOR SQLSTATE '23000'
        CALL p_log_test('FK02', 'FK02: hobby_id debe existir', 'PASS');
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        CALL p_log_test('FK02', 'ERROR: No se produjo el error esperado
para hobby_id', 'ERROR');

    CALL p_populate();

    INSERT INTO user_hobby_links (user_id, hobby_id) VALUES (1, 999);

    CALL p_log_test('FK02', 'ERROR: Se permitió FK hobby_id inexistente',
'FAIL');
END //
DELIMITER ;

-- =====
-- ORQUESTADOR
-- =====

DELIMITER //
CREATE OR REPLACE PROCEDURE p_run_tests()
BEGIN

```

```

-- Restaurar la población positiva antes de ejecutar los casos
negativos.
DECLARE EXIT HANDLER FOR SQLEXCEPTION
BEGIN
    CALL p_log_test('POPULATE', 'ERROR: El populate falló. No se
ejecutaron los tests negativos.', 'ERROR');
    SELECT * FROM test_results ORDER BY execution_time, test_id;
    SELECT test_status, COUNT(*) AS total FROM test_results GROUP BY
test_status;
END;

DELETE FROM test_results;

CALL p_populate();

CALL p_test_rn01_age_adult();
CALL p_test_rn02_unique_email();
CALL p_test_rn03_unique_hobby_per_user();
CALL p_test_fk_user_exists();
CALL p_test_fk_hobby_exists();

SELECT * FROM test_results ORDER BY execution_time, test_id;
SELECT test_status, COUNT(*) AS total FROM test_results GROUP BY
test_status;
END //
DELIMITER ;

CALL p_run_tests();

```

i Info

Versión PDF disponible